

1 Solution — GIAM 2.1

Problem 5

1.1 Problem

The Sheffer stroke $|$ (**NAND**) and the Peirce arrow $\downarrow$ (**NOR**) are defined by

$$A | B = \neg(A \wedge B), \quad A \downarrow B = \neg(A \vee B).$$

Find an expression for $(A \wedge \neg B) \vee C$ using only these connectives (together with negation and the variables).

1.2 Answer

$$(A \wedge \neg B) \vee C = \neg(\neg(A | \neg B) \downarrow C).$$

1.3 The Derivation

The two definitions, read backwards, let us *replace* $\wedge$ and $\vee$:

$$A \wedge B = \neg(A | B), \quad A \vee B = \neg(A \downarrow B).$$

Apply these to the target, outermost connective first.

Step 1 — rewrite the outer $\vee$ as a NOR. With $X = A \wedge \neg B$,

$$(A \wedge \neg B) \vee C = \neg((A \wedge \neg B) \downarrow C).$$

Step 2 — rewrite the inner $\wedge$ as a NAND. Using $A \wedge \neg B = \neg(A | \neg B)$,

$$(A \wedge \neg B) \vee C = \neg(\neg(A | \neg B) \downarrow C).$$

This uses only $|$, $\downarrow$, $\neg$, and the variables — as required.

Rewrite $\vee$ and $\wedge$ using the two identities, then substitute. Negation is allowed.

NAND: $A \mid B = \neg(A \wedge B)$

so $A \wedge B = \neg(A \mid B)$
(true unless both true)

NOR: $A \downarrow B = \neg(A \vee B)$

so $A \vee B = \neg(A \downarrow B)$
(true only if both false)

Start: $(A \wedge \neg B) \vee C$

$\downarrow$ turn the outer $\vee$ into NOR: $X \vee C = \neg(X \downarrow C)$

$= \neg((A \wedge \neg B) \downarrow C)$

$\downarrow$ turn the $\wedge$ into NAND: $A \wedge \neg B = \neg(A \mid \neg B)$

$= \neg(\neg(A \mid \neg B) \downarrow C)$

$(A \wedge \neg B) \vee C = \neg(\neg(A \mid \neg B) \downarrow C)$

Verification (all 8 rows agree):

A B C	$A \mid \neg B$	$\neg(A \mid \neg B)$	$\dots \downarrow C$	$\neg(\dots) = \text{target}$
1 1 1	1	0	0	1
1 1 0	1	0	1	0
1 0 1	0	1	0	1
1 0 0	0	1	0	1
0 ..	1	0	$\neg C$	C

Figure 1.1: The two identities are shown: NAND $A \mid B = \text{not}(A \text{ and } B)$ so $A \text{ and } B = \text{not}(A \mid B)$; NOR $A \downarrow B = \text{not}(A \text{ or } B)$ so $A \text{ or } B = \text{not}(A \downarrow B)$. The reduction: start with $(A \text{ and not } B) \text{ or } C$; turn the outer or into NOR giving $\text{not}((A \text{ and not } B) \downarrow C)$; turn the and into NAND giving $\text{not}(\text{not}(A \mid \text{not } B) \downarrow C)$, the boxed answer. A verification table shows all eight rows agree: for $A=1, B=1$ the target is C ; for $A=1, B=0$ the target is 1; and for $A=0$ the target is C — matching in every case.

1.4 Verification

Tracing the built expression against the target on all eight rows (writing 1 for true, 0 for false):

A	B	C	$A \mid \neg B$	$\neg(A \mid \neg B)$	$\neg(A \mid \neg B) \downarrow C$	$\neg(\dots)$	target
1	1	1	1	0	0	1	1
1	1	0	1	0	1	0	0
1	0	1	0	1	0	1	1
1	0	0	0	1	0	1	1

0	1	1	1	0	0	1	1
0	1	0	1	0	1	0	0
0	0	1	1	0	0	1	1
0	0	0	1	0	1	0	0

Table 1.1

The last two columns match in every row, confirming the expression. ■

1.5 Remark — The General Recipe: NAND for $\wedge$, NOR for $\vee$

The derivation followed a mechanical rule that works for *any* formula: process it from the outside in, and each time you meet an $\wedge$ or $\vee$, replace it using

$$X \wedge Y \rightsquigarrow \neg(X \mid Y), \quad X \vee Y \rightsquigarrow \neg(X \downarrow Y).$$

Because $\mid$ and $\downarrow$ are literally “negated-and” and “negated-or,” each swap costs exactly one outer negation. Since negation is permitted here, *any* sentence built from $\wedge, \vee, \neg$ can be translated this way — the pair $\{\mid, \downarrow, \neg\}$ is **functionally complete**. The translation is not unique; one could equally rewrite the outer $\vee$ via NAND and De Morgan, giving a different but logically identical expression.

1.6 Remark — Negation Is Redundant: One Connective Suffices

The problem allows $\neg$ as a convenience, but it is not actually needed — **NAND alone is functionally complete**, and so is **NOR alone**. The reason is that each can build its own negation:

$$\neg X = X \mid X = X \downarrow X.$$

(Check: $X \mid X = \neg(X \wedge X) = \neg X$, and $X \downarrow X = \neg(X \vee X) = \neg X$.) Feeding a wire into *both* inputs of a NAND gate inverts it. Substituting $B \mid B$ for every $\neg B$ (and likewise for the outer negations) turns our answer into a NAND-*only* expression; the same trick with $\downarrow$ gives a NOR-only one. Both were checked to reproduce the target on all

eight rows. This is exactly why real hardware can be — and often is — fabricated from a single gate type.

1.7 Remark — This Is Why NAND/NOR Rule Chip Design

The functional completeness of NAND (and NOR) is not just a logician’s curiosity — it is the organizing principle of digital hardware. A chip fabricator who can reliably etch *one* kind of gate can build *every* Boolean function, hence any circuit, from copies of it. NAND and NOR are the natural choices because a single CMOS NAND or NOR needs only a handful of transistors and inverts “for free” (the transistor stages are naturally inverting). So a memory cell, an adder, a whole processor, can be laid down as a sea of identical NAND gates. The XOR of Problem 1, for instance, is customarily built from four NAND gates. This exercise — reducing an everyday $\wedge/\vee$ formula to strokes and arrows — is the pencil-and-paper version of that entire design philosophy.

1.8 Remark — Sheffer’s and Peirce’s Discovery

That a single connective could generate all of logic was found by **Charles Sanders Peirce** around 1880 (for NOR, his “ampheck”) and independently published by **Henry Sheffer** in 1913 (for NAND, the “stroke”). The result was striking because the standard development of logic used several primitives ($\neg, \wedge, \vee, \rightarrow$); Sheffer showed one binary operation sufficed to define them all. It is a logical echo of a recurring theme in this book — reducing a rich system to the fewest possible primitives, as with the axioms of arithmetic or the generators of a group. Here the “fewest possible” is exactly **one**: every truth function whatsoever is some tower of NANDs.